Performance & Optimization in ASP.NET Core Web API

---

1. Introduction to Performance Optimization

- Performance = how fast and efficiently an API responds
- Optimization improves:
  - Response time
  - Throughput (requests/sec)
  - Resource usage (CPU, memory)

---

2. Asynchronous Programming (Async/Await)

Why use async?

- Prevents thread blocking
- Improves scalability

```
public async Task<IActionResult> GetProducts()
{
   var products = await _context.Products.ToListAsync();
   return Ok(products);
}
```

✓ **Key Points:**

- Use async and await for I/O operations
- Avoid blocking calls like .Result or .Wait()

---

3. Caching Techniques

→ Caching is a technique that stores frequently accessed data in a fast-access storage layer (typically memory) to reduce server load and latency.

→ Instead of recalculating expensive operations or querying a database for every request, the application serves the data from the cache.

Types of Caching:

1. In-Memory Caching

services.AddMemoryCache();

- Stores data in server memory
- Fast but not shared across servers

---

2. Response Caching

[ResponseCache(Duration = 60)]

- Caches HTTP responses
- Reduces server load

Note: 3rd party libraires like Redis helps for scalable applications across networks.

✓ **Benefits:**

- Faster responses
- Reduced database calls

---

4. Compression

- Reduces response size

```
builder.Services.AddResponseCompression();
```

Common formats: Gzip, Brotli

---

5. Efficient Data Access

Use Entity Framework Core efficiently:

```
// Use AsNoTracking() for read-only queries
_context.Products.AsNoTracking().ToListAsync();

// Select only required fields:
_context.Products.Select(p => new { p.Name, p.Price });

// Use pagination (Skip, Take)
```

---

6. Pagination

```
var data = _context.Products
    .Skip((page - 1) * pageSize)
    .Take(pageSize);
```

✓ **Benefits:**

- Reduces payload size
- Improves performance

---

7. Rate Limiting

- Limits number of requests per client

✓ **Prevents:**

- Abuse
- Server overload

---

8. Load Balancing

- Distributes traffic across multiple servers

✓ **Benefits:**

- High availability
- Better performance

---

9. Connection Pooling

- Reuses database connections
- Managed automatically in EF Core

✓ **Benefit:**

- Reduces connection overhead

---

10. Logging Optimization

- Avoid excessive logging
- Use appropriate log levels:
  - Information
  - Warning
  - Error

---

11. Minimizing Middleware Overhead

- Only use required middleware
- Order middleware correctly

---

12. Use of DTOs (Data Transfer Objects)

- Avoid sending full entity models

```
public class ProductDTO
{
   public string Name { get; set; }
}
```

✓ **Benefits:**

- Smaller response size
- Better security

---

13. HTTP/2 and HTTPS

- HTTP/2 improves performance via:
  - Multiplexing
  - Header compression

---

14. Profiling and Monitoring Tools

Tools:

- Application Insights
- dotTrace
- MiniProfiler

---

15. Avoid Over-fetching & Under-fetching

- Over-fetching → too much data
- Under-fetching → too many requests

✓ **Solution:**

- Use optimized queries
- Use filtering

Best Practices Summary

- Use async/await for I/O
- Implement caching
- Optimize database queries
- Use pagination & DTOs
- Enable compression
- Monitor performance regularly

Quick Revision for Interview

- **Async/Await** → improves scalability
- **Caching** → reduces database load
- **Pagination** → reduces response size
- **Compression** → reduces payload
- **DTOs** → optimize data transfer
- **Rate limiting** → prevents overload